

```
!pip install torch torchvision
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import glob
import matplotlib.pyplot as plt
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.24.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.9)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.9)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
```

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
EPOCHS = 20
BATCH_SIZE = 32
LR = 0.0002
IMG_SIZE = 128
DATA_PATH = "/content/monet2photo/trainA"
```

```
print("Using Device:", DEVICE)
```

```
Using Device: cuda
```

```
class MonetDataset(Dataset):
    def __init__(self, path, transform=None, max_images=500):
        self.images = glob.glob(path + "/*.jpg")[:max_images]
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img = Image.open(self.images[idx]).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img
```

```
import os
print(os.listdir("/content"))
```

```
['.config', 'sample_data']
```

```
!wget https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/monet2photo.zip
!unzip -q monet2photo.zip
```

```
--2026-02-25 13:46:45-- https://efrosgans.eecs.berkeley.edu/cyclegan/datasets/monet2photo.zip
Resolving efrosgans.eecs.berkeley.edu (efrosgans.eecs.berkeley.edu)... 128.32.244.190
Connecting to efrosgans.eecs.berkeley.edu (efrosgans.eecs.berkeley.edu)|128.32.244.190|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 305231073 (291M) [application/zip]
Saving to: 'monet2photo.zip'

monet2photo.zip  100%[=====>] 291.09M  71.9MB/s   in 3.9s

2026-02-25 13:46:49 (74.3 MB/s) - 'monet2photo.zip' saved [305231073/305231073]
```

```
import os
print(os.listdir("/content"))
```

```
['.config', 'monet2photo', 'monet2photo.zip', 'sample_data']
```

```
print(len(os.listdir("/content/monet2photo/trainA")))
```

```
1072
```

```
import glob
from PIL import Image
from torch.utils.data import Dataset

class MonetDataset(Dataset):
    def __init__(self, path, transform=None, max_images=500):
        self.images = glob.glob(path + "/*")[:max_images]
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img = Image.open(self.images[idx]).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img
```

```
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

IMG_SIZE = 128
BATCH_SIZE = 32
DATA_PATH = "/content/monet2photo/trainA"

transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.CenterCrop(IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

dataset = MonetDataset(DATA_PATH, transform=transform, max_images=500)
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

print("Total Images Loaded:", len(dataset))
```

```
Total Images Loaded: 500
```

```
import torch
import torch.nn as nn

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.ConvTranspose2d(100, 512, 4, 1, 0, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(True),

            nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
```

```

        nn.BatchNorm2d(256),
        nn.ReLU(True),

        nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),
        nn.BatchNorm2d(128),
        nn.ReLU(True),

        nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),
        nn.BatchNorm2d(64),
        nn.ReLU(True),

        nn.ConvTranspose2d(64, 3, 4, 2, 1, bias=False),
        nn.Tanh()
    )

    def forward(self, x):
        return self.model(x)

```

```

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, True),

            nn.Conv2d(64, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, True),

            nn.Conv2d(128, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, True),

            nn.Conv2d(256, 512, 4, 2, 1, bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, True),

            nn.Conv2d(512, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

```

```

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

G = Generator().to(DEVICE)
D = Discriminator().to(DEVICE)

import torch.optim as optim

LR = 0.0002

criterion = nn.BCELoss()
optimizer_G = optim.Adam(G.parameters(), lr=LR, betas=(0.5,0.999))
optimizer_D = optim.Adam(D.parameters(), lr=LR, betas=(0.5,0.999))

print("Models Initialized Successfully ✅")

```

Models Initialized Successfully ✅

```

import torch
print("CUDA Available:", torch.cuda.is_available())
print("Device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")

CUDA Available: True
Device: Tesla T4

```

```

EPOCHS = 20

for epoch in range(EPOCHS):
    for real_images in loader:

        real_images = real_images.to(DEVICE)
        batch_size = real_images.size(0)

```

```

# ---- Train Discriminator ----
optimizer_D.zero_grad()

outputs_real = D(real_images)
real_labels = torch.ones_like(outputs_real).to(DEVICE)
loss_real = criterion(outputs_real, real_labels)

noise = torch.randn(batch_size,100,1,1).to(DEVICE)
fake_images = G(noise)

outputs_fake = D(fake_images.detach())
fake_labels = torch.zeros_like(outputs_fake).to(DEVICE)
loss_fake = criterion(outputs_fake, fake_labels)

loss_D = loss_real + loss_fake
loss_D.backward()
optimizer_D.step()

# ---- Train Generator ----
optimizer_G.zero_grad()

outputs = D(fake_images)
real_labels = torch.ones_like(outputs).to(DEVICE)
loss_G = criterion(outputs, real_labels)

loss_G.backward()
optimizer_G.step()

print("Epoch [{epoch+1}/{EPOCHS}]  Loss_D: {loss_D.item():.4f}  Loss_G: {loss_G.item():.4f}")

print("DCGAN Training Finished ✅")

```

```

Epoch [1/20]  Loss_D: 0.0003  Loss_G: 9.0520
Epoch [2/20]  Loss_D: 0.0003  Loss_G: 9.1354
Epoch [3/20]  Loss_D: 0.0004  Loss_G: 9.2214
Epoch [4/20]  Loss_D: 0.0003  Loss_G: 9.1638
Epoch [5/20]  Loss_D: 0.0003  Loss_G: 9.5163
Epoch [6/20]  Loss_D: 0.0002  Loss_G: 9.6117
Epoch [7/20]  Loss_D: 0.0003  Loss_G: 9.4904
Epoch [8/20]  Loss_D: 0.0002  Loss_G: 9.7266
Epoch [9/20]  Loss_D: 0.0002  Loss_G: 9.6877
Epoch [10/20] Loss_D: 0.0002  Loss_G: 9.8369
Epoch [11/20] Loss_D: 0.0002  Loss_G: 9.5818
Epoch [12/20] Loss_D: 0.0003  Loss_G: 9.9599
Epoch [13/20] Loss_D: 0.0001  Loss_G: 9.8692
Epoch [14/20] Loss_D: 0.0001  Loss_G: 9.9644
Epoch [15/20] Loss_D: 0.0001  Loss_G: 9.9870
Epoch [16/20] Loss_D: 0.0001  Loss_G: 9.9510
Epoch [17/20] Loss_D: 0.0001  Loss_G: 10.0846
Epoch [18/20] Loss_D: 0.0001  Loss_G: 10.0921
Epoch [19/20] Loss_D: 0.0001  Loss_G: 10.2310
Epoch [20/20] Loss_D: 0.0001  Loss_G: 10.2352
DCGAN Training Finished ✅

```

```

import matplotlib.pyplot as plt
import torchvision
import torch

G.eval()

# Get 5 real images
real_images = next(iter(loader))[:5]
real_images = real_images.cpu()

# Generate 5 fake images
noise = torch.randn(5,100,1,1).to(DEVICE)
with torch.no_grad():
    fake_images = G(noise).cpu()

# Denormalize
real_images = real_images * 0.5 + 0.5
fake_images = fake_images * 0.5 + 0.5

plt.figure(figsize=(10,6))

for i in range(5):

```

```
# Real image
plt.subplot(5,2,2*i+1)
plt.imshow(real_images[i].permute(1,2,0))
plt.title("Real Monet")
plt.axis("off")

# Generated image
plt.subplot(5,2,2*i+2)
plt.imshow(fake_images[i].permute(1,2,0))
plt.title("Generated Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Real Monet



Generated Image



Real Monet



Generated Image



Real Monet



Generated Image



Real Monet



Generated Image



Real Monet



Generated Image



Start coding or [generate](#) with AI.